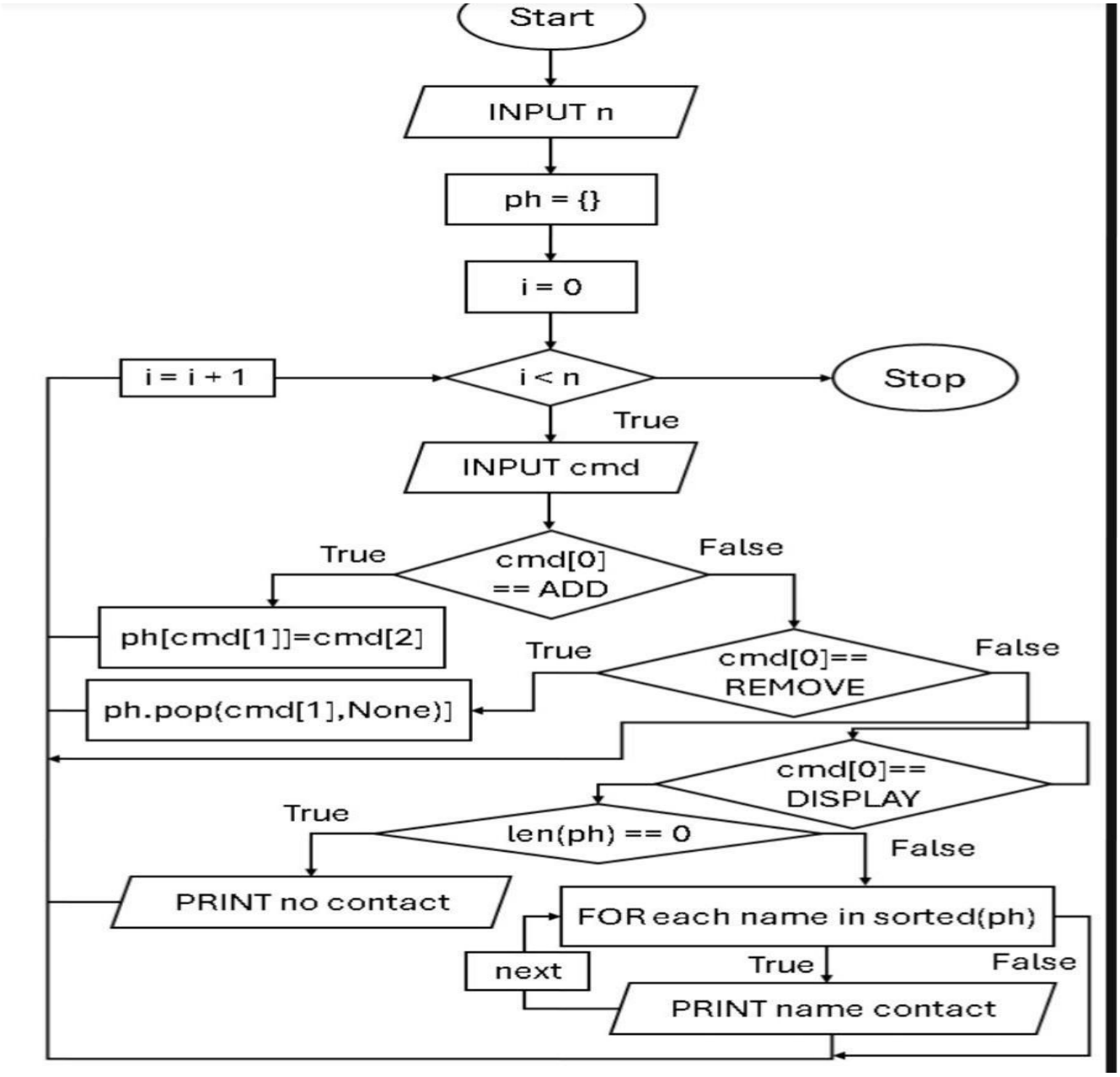
12.1.1 Phone Number Database Management

Algorithm :

```
1. Initialize empty dictionary contacts = {}
2. Read integer n (number of operations)
3. For i = 1 to n:
   a. Read operation line and split into words
   b. Identify operation type (first word):

      Case "ADD":
        - Extract name (2nd word) and phone (3rd word)
        - Add/Update entry: contacts[name] = phone

      Case "REMOVE":
        - Extract name (2nd word)
        - If name exists in contacts:
          Delete the entry contacts[name]

      Case "DISPLAY":
        - If contacts dictionary is empty:
          Print "No contacts"
        - Else:
          Get all names from contacts
          Sort names alphabetically
          For each name in sorted list:
            Print "name: phone_number"

4. End
```

\

Flowchart :


Start
INPUT n
ph = {}
i = 0
i = i + 1
i < n
Stop
True
INPUT cmd
True
cmd[0]
== ADD
False
ph[cmd[1]]=cmd[2]
True
cmd[0]==
REMOVE
False
ph.pop(cmd[1],None)]
cmd[0]==
DISPLAY
True
len(ph) == 0
False
PRINT no contact
FOR each name in sorted(ph)
next
True
False
PRINT name contact

CODE:

```
contacts = {}
n = int(input())
for _ in range(n):
    operation = input().split()

    if operation[0] == "ADD":
        name = operation[1]
        phone = operation[2]
        contacts[name] = phone
    elif operation[0] == "REMOVE":
        name = operation[1]
        if name in contacts:
            del contacts[name]

    elif operation[0] == "DISPLAY":
        if not contacts:
            print("No contacts")
        else:
            for name in sorted(contacts):
                print(f"{name}: {contacts[name]}")
```

12.1.1. Phone Number Database Management

03:00

Write a Python program to manage a phone number database using a dictionary. The dictionary stores contact names as keys and phone numbers as values.

The program should support three operations: adding a contact (if the name already exists, update the phone number), removing a contact (if the name does not exist, ignore the operation), and displaying all contacts in alphabetical order by name.

Input Format:

1. First line contains an integer n representing the number of operations.
2. Each of the next n lines contains a single operation in one of the following formats:
   - "ADD <name> <phone_number>" where <name> is a string, <phone_number> is a string of 10 digits.
   - "REMOVE <name>" - name is a string.
   - "DISPLAY" - no additional parameters.

Output Format:

- For display operation, print all contacts in alphabetical order by name in the format:

```
<name>: <phone_number>
```

- If the database is empty during display operation, print: "No contacts".
- Do not print anything for add and remove operations.

Constraints:

- Name contains only alphabetic characters (no spaces). And it is case sensitive.

Sample Test Cases +

Explorer

phoneDat...

Submit

Debugger

```python
contacts = {}
n = int(input())
for _ in range(n):
    operation = input().split()

    if operation[0] == "ADD":
        name = operation[1]
        phone = operation[2]
        contacts[name] = phone
    elif operation[0] == "REMOVE":
        name = operation[1]
        if name in contacts:
            del contacts[name]
```

Average time	Maximum time
0.011 s	0.015 s
11.25 ms	15.00 ms

5 out of 5 shown test case(s) passed

3 out of 3 hidden test case(s) passed

- Test case 1 14 ms
- Test case 2 10 ms
- Test case 3 11 ms
- Test case 4 11 ms

Terminal Test cases